

Sunset Temperature Prediction at Rubin Observatory

Machine Learning Approaches for 3-Hour Forecasting

Analysis Report

January 2026

Abstract

We present a comprehensive analysis of algorithms for predicting exterior temperature at sunset at the Vera C. Rubin Observatory on Cerro Pachón, Chile. Using approximately two years of 15-minute interval temperature data (October 2023–December 2025), we developed and evaluated multiple prediction approaches including traditional methods and machine learning models. Models were **trained on odd calendar days** and **tested on even calendar days** to ensure rigorous out-of-sample evaluation. Our best model (Random Forest with 3-day lookback) achieves an **RMS error of 1.01°C** at 3 hours before sunset with **bias of only −0.04°C**, representing a 61% improvement over the persistence baseline.

1 Introduction

Accurate prediction of sunset temperature is critical for telescope operations, particularly for thermal equilibration of optical components. This report documents our development of a prediction system with the following objectives:

- **Primary metric:** Minimize RMS error at 3 hours before sunset
- **Secondary goal:** Minimize systematic bias
- **Validation strategy:** Train on odd calendar days, test on even calendar days

1.1 Data Overview

- **Location:** Cerro Pachón, Chile (30°14'40" S, 70°44'58" W)
- **Time period:** October 2023 – December 2025
- **Total valid days:** 771 days
- **Temporal resolution:** 15-minute intervals
- **Timestamps:** UTC (determined from diurnal cycle analysis)

1.2 Train/Test Split Strategy

To ensure rigorous out-of-sample evaluation, we split the data by calendar day number:

- **Training set (odd days):** 394 days (days 1, 3, 5, 7, ... of each month)
- **Testing set (even days):** 377 days (days 2, 4, 6, 8, ... of each month)

This interleaved split ensures that training and testing data span the same time periods and seasonal conditions, eliminating temporal bias while maintaining strict separation. Figure 1 illustrates the data distribution.

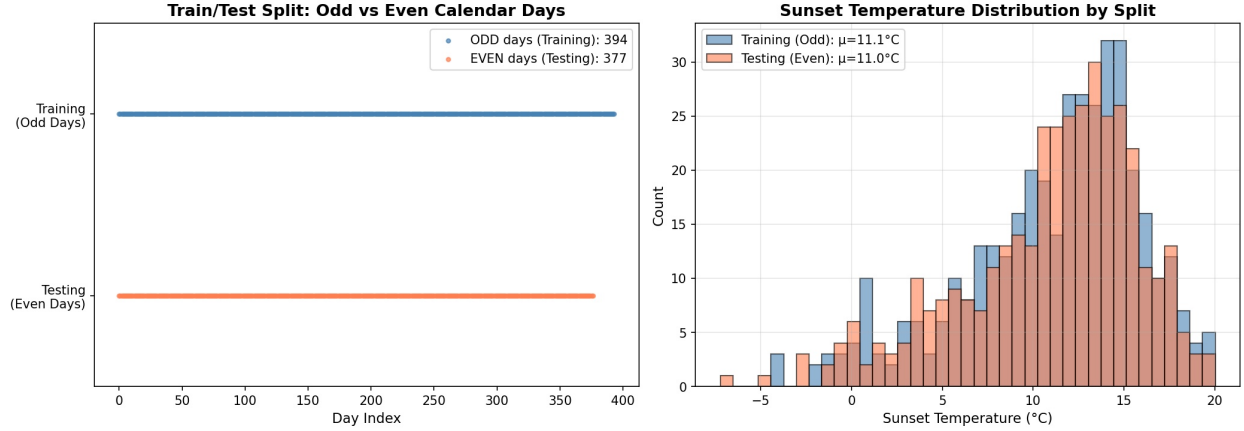


Figure 1: **Left:** Visualization of odd/even day split across the dataset. **Right:** Sunset temperature distributions for training (odd days, blue) and testing (even days, orange) sets, showing similar statistical properties.

2 Development History

2.1 Phase 1: Baseline Models

Initial implementation included:

1. **Persistence:** Current temperature as prediction ($\text{RMS} = 2.57^{\circ}\text{C}$ at 3h)
2. **Historical Mean:** Seasonal average (± 15 day window) ($\text{RMS} = 3.60^{\circ}\text{C}$)
3. **Linear Extrapolation:** 2-3 hour trend projection ($\text{RMS} = 2.74\text{--}3.01^{\circ}\text{C}$)
4. **Fourier Decomposition:** 24h, 12h, 6h, 3h harmonics with polynomial baseline

Finding: Fourier methods underperformed persistence due to non-sinusoidal temperature evolution near sunset.

2.2 Phase 2: Machine Learning Models

Implemented ensemble and neural network approaches:

- Random Forest, Gradient Boosting, XGBoost, HistGradientBoosting
- Support Vector Regression (RBF kernel)
- Multi-layer Perceptron (MLP)
- Ridge regression

Features: Current and lookback temperatures (0.5–4h), temperature trends, daily statistics (max, min, range), and seasonal encoding (sin/cos of day-of-year).

Result: Random Forest achieved **1.08°C RMS**—a 58% improvement over persistence.

2.3 Phase 3: Delta Prediction and Bias Correction

Key improvements:

1. **Delta prediction:** Predict temperature *change* from current to sunset, rather than absolute temperature

2. **Calibration-based bias correction:** Use held-out calibration set (15% of training data) to measure and subtract systematic bias

2.4 Phase 4: Lookback Optimization

Investigated optimal historical data window. **Finding:** 3 days (72 hours) of lookback is optimal, as integral day boundaries correctly capture diurnal temperature cycles.

Final feature set includes: previous days' mean temperatures, temperature ranges, and sunset temperatures.

3 Results

3.1 Performance Overview

Figure 2 shows RMS prediction error across all models and lead times.

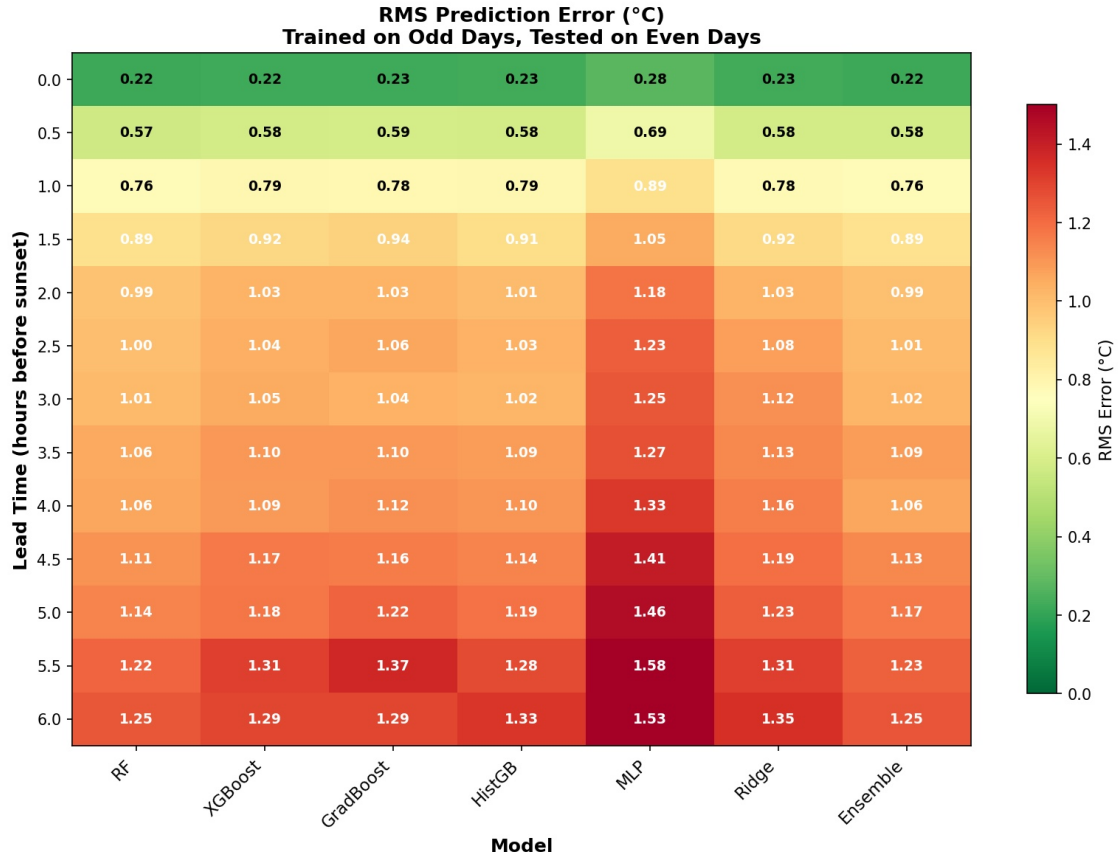


Figure 2: RMS prediction error (°C) by model and lead time. Green indicates lower error. Random Forest and Ensemble models achieve the best performance, with $\text{RMS} \approx 1.0^\circ\text{C}$ at 3 hours.

3.2 Multiple Error Metrics

Figure 3 presents median and mean absolute error heatmaps.

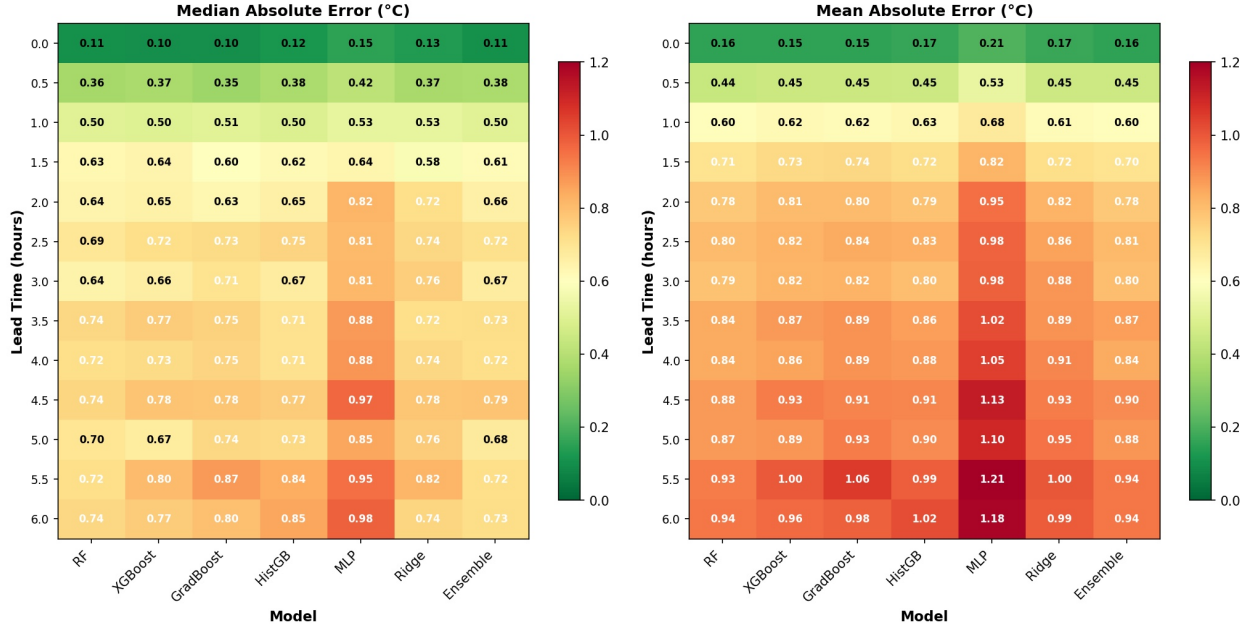


Figure 3: **Left:** Median absolute error. **Right:** Mean absolute error. The median is consistently lower than the mean, indicating a right-skewed error distribution with occasional large outliers.

3.3 Error Distributions

Figure 4 shows prediction error histograms at 3 hours lead time.

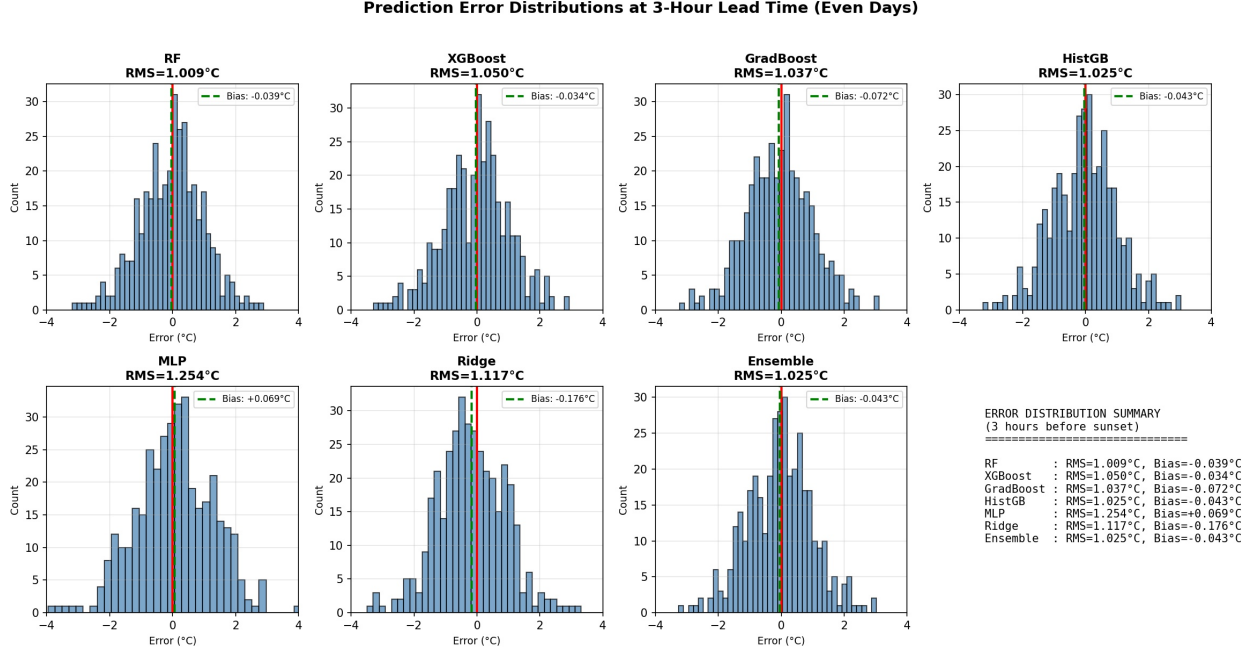


Figure 4: Error distribution histograms at 3-hour lead time for all models tested on even days. Red vertical line indicates zero error; green dashed line shows mean bias. Random Forest shows the tightest distribution with minimal bias (-0.04°C).

3.4 Cumulative Distribution Functions

Figure 5 presents cumulative probability distributions of absolute errors.

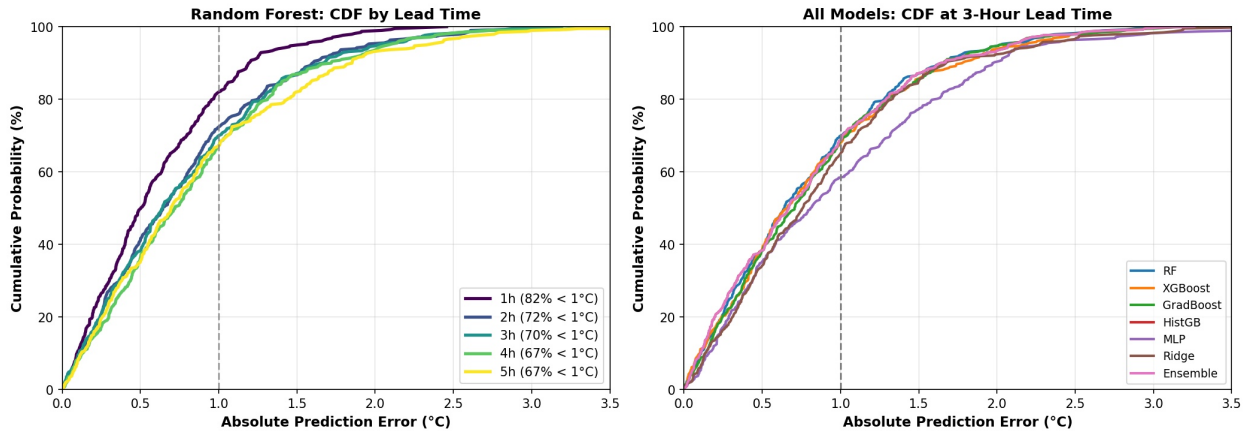


Figure 5: **Left:** CDF for Random Forest at different lead times, showing 70% of 3-hour predictions have error $<1^{\circ}\text{C}$. **Right:** CDF comparison across all models at 3 hours.

3.5 RMS vs Lead Time

Figure 6 shows how prediction error grows with lead time.

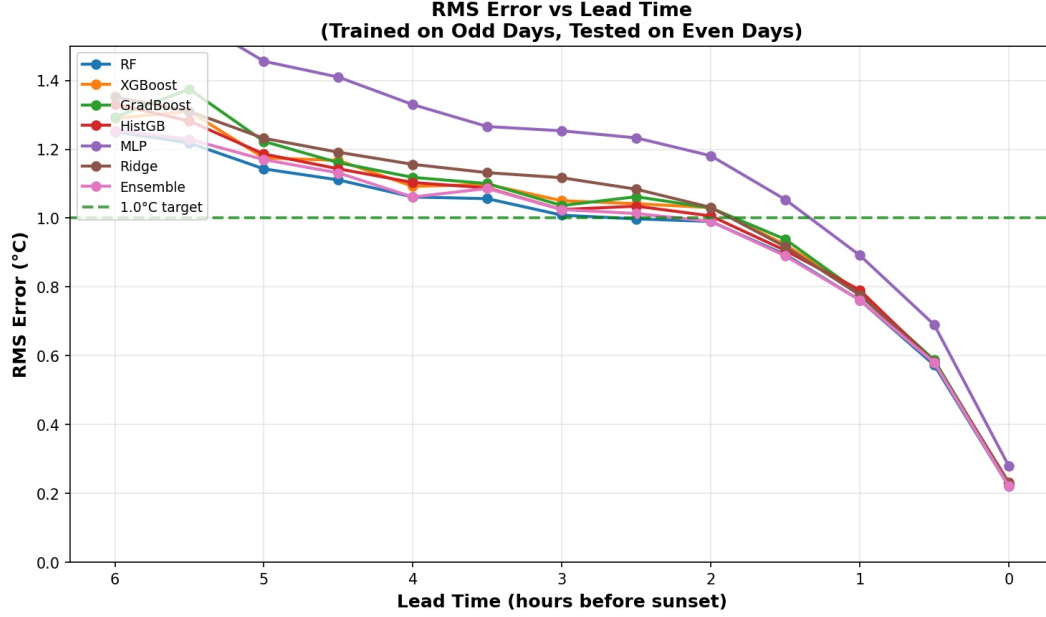


Figure 6: RMS error vs lead time for all models. Error grows approximately linearly at $\sim 0.15^{\circ}\text{C}$ per hour. At 3 hours, best models achieve $\text{RMS} \approx 1.0^{\circ}\text{C}$.

3.6 Summary Table

Table 1 presents final performance metrics at 3 hours before sunset.

Table 1: Model Performance at 3-Hour Lead Time (Tested on Even Days)

Model	RMS ($^{\circ}\text{C}$)	MAE ($^{\circ}\text{C}$)	Median ($^{\circ}\text{C}$)	Bias ($^{\circ}\text{C}$)	% < 1°C
Random Forest	1.009	0.791	0.643	-0.039	69.8
Ensemble	1.025	0.799	0.669	-0.043	68.4
HistGradientBoosting	1.025	0.799	0.669	-0.043	68.4
GradientBoosting	1.037	0.821	0.707	-0.072	68.4
XGBoost	1.050	0.820	0.662	-0.034	67.9
Ridge	1.117	0.882	0.761	-0.176	64.7
MLP	1.254	0.982	0.814	+0.069	58.4
Persistence (baseline)	2.571	2.307	—	-2.244	—

4 Error Analysis

4.1 Temporal Evolution

Figure 7 shows how prediction errors evolve over the two-year dataset.

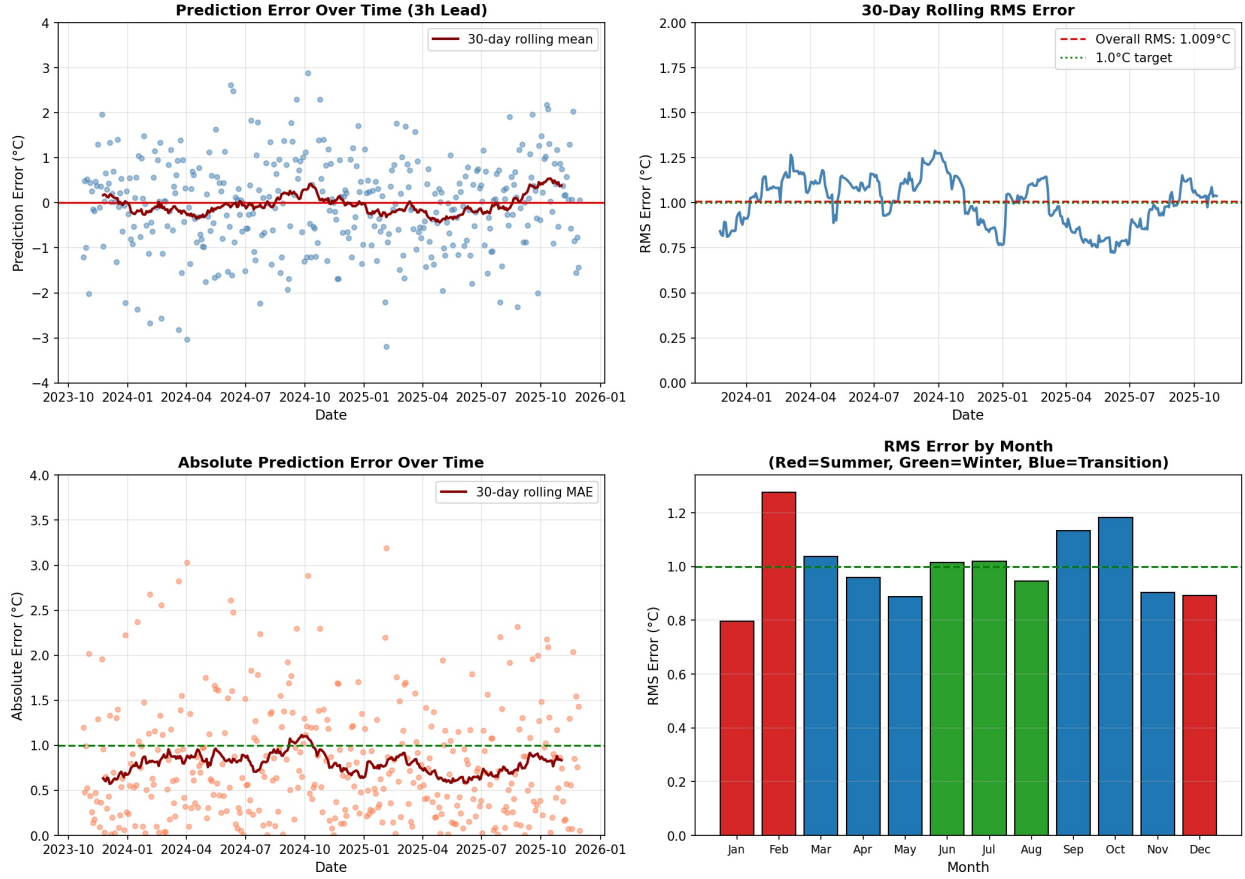


Figure 7: Temporal evolution of prediction errors. **Top left:** Individual errors over time with 30-day rolling mean. **Top right:** 30-day rolling RMS showing stable performance. **Bottom left:** Absolute errors over time. **Bottom right:** Monthly RMS (red=summer, green=winter, blue=transition seasons).

4.2 Seasonal Effects

Figure 8 analyzes performance differences across seasons.

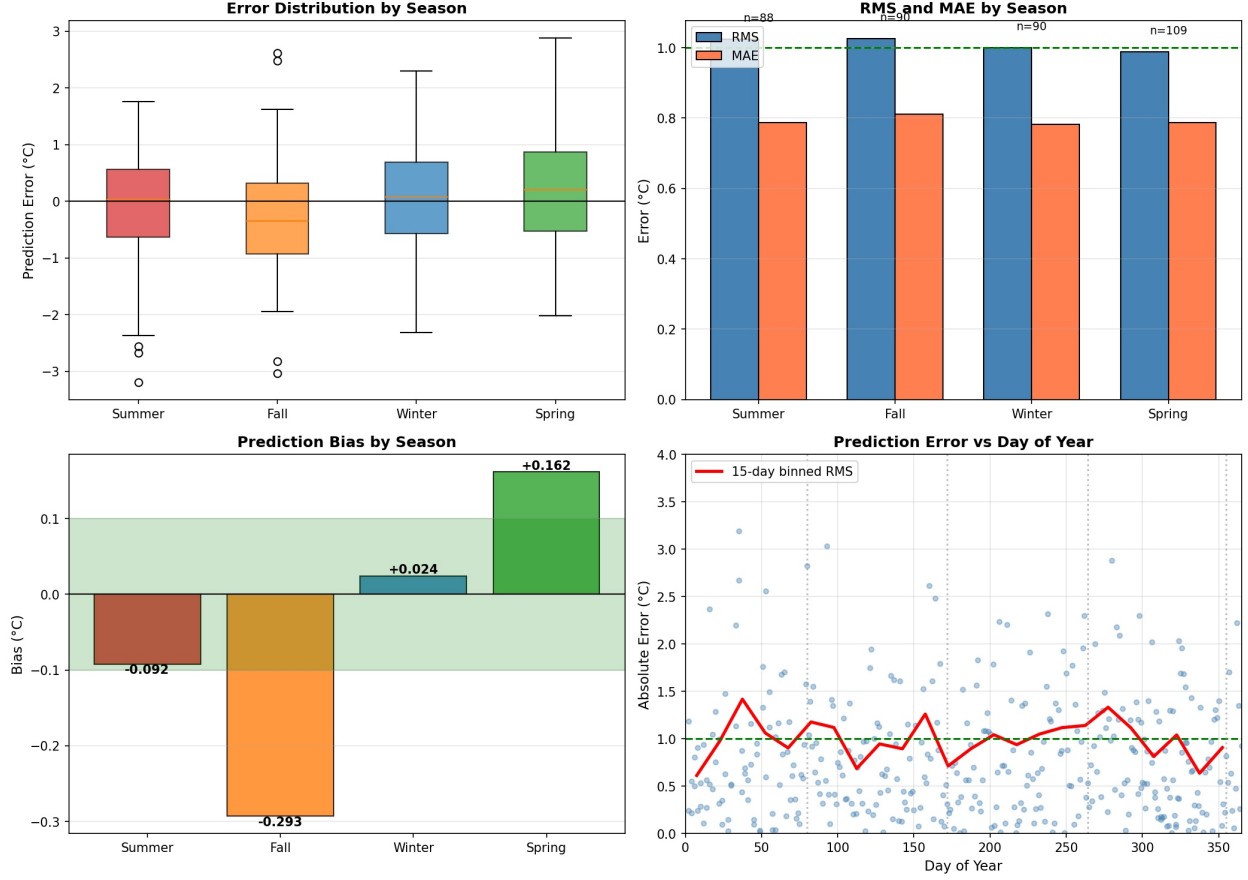


Figure 8: Seasonal analysis. **Top left:** Error distribution by season. **Top right:** RMS and MAE by season. **Bottom left:** Bias by season—Fall shows largest negative bias (-0.29°C), Spring shows positive bias ($+0.16^{\circ}\text{C}$). **Bottom right:** Error vs day of year with 15-day binned RMS.

Table 2 summarizes seasonal performance metrics.

Table 2: Seasonal Performance Summary

Season	RMS ($^{\circ}\text{C}$)	MAE ($^{\circ}\text{C}$)	Bias ($^{\circ}\text{C}$)	N days
Summer (Dec–Feb)	1.024	0.788	-0.092	88
Fall (Mar–May)	1.026	0.810	-0.293	90
Winter (Jun–Aug)	1.000	0.782	$+0.024$	90
Spring (Sep–Nov)	0.988	0.786	$+0.162$	109

Key observation: Seasonal bias varies from -0.29°C (Fall) to $+0.16^{\circ}\text{C}$ (Spring). This suggests that season-specific bias correction could further improve performance.

4.3 Dependence on Temperature Trends

Figure 9 examines whether errors depend on multi-day temperature trends.

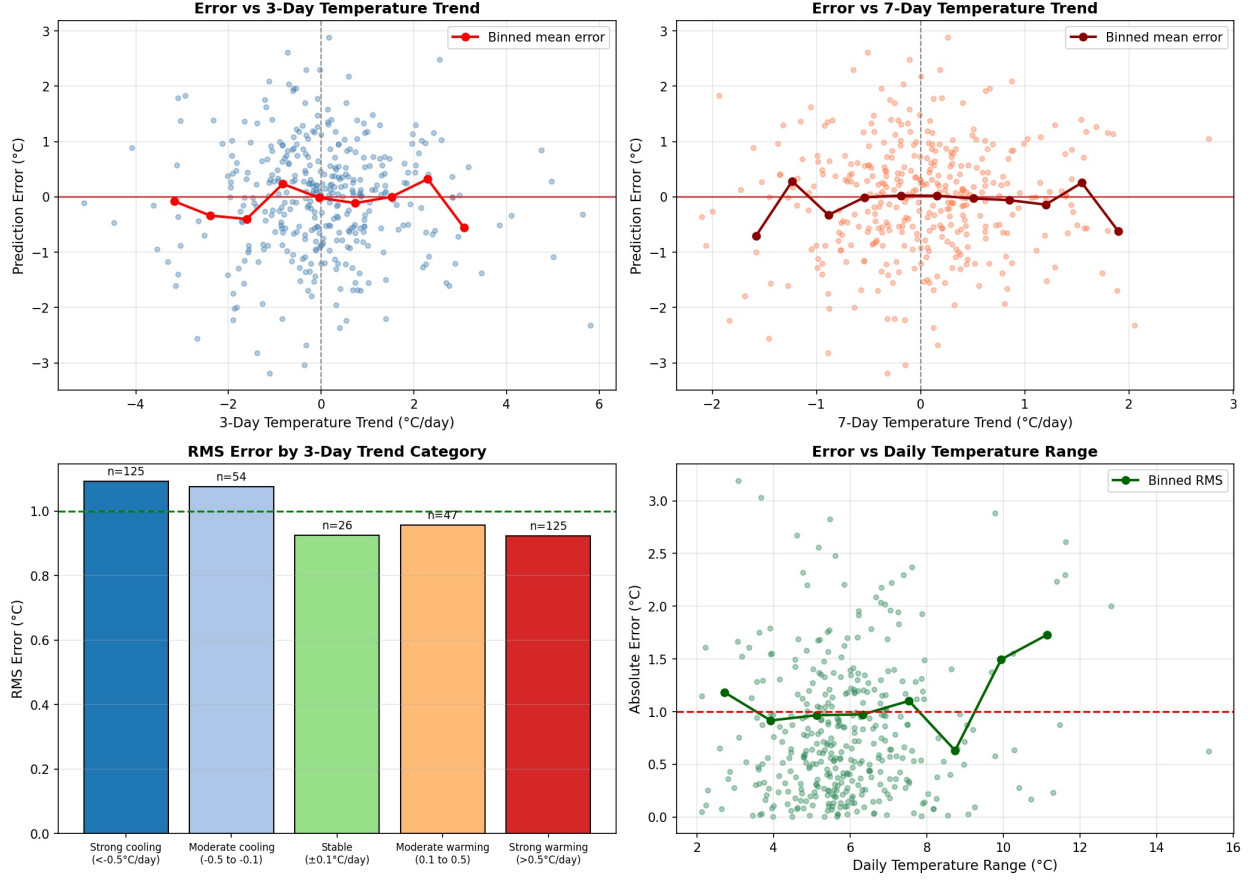


Figure 9: Error dependence on temperature trends. **Top left:** Error vs 3-day trend. **Top right:** Error vs 7-day trend. **Bottom left:** RMS by trend category—stable conditions ($\pm 0.1^\circ\text{C}/\text{day}$) show lowest RMS (0.93°C). **Bottom right:** Error vs daily temperature range.

Key findings:

- Correlations between temperature trends and errors are weak ($r < 0.1$)
- **Stable days** (trend $\pm 0.1^\circ\text{C}/\text{day}$) show lowest RMS: 0.93°C
- **Strong cooling** days ($< -0.5^\circ\text{C}/\text{day}$) show highest RMS: 1.09°C
- Days with larger diurnal temperature range show slightly higher errors ($r = 0.10$)

5 Experimental: LLM-Style Embedding Approach

As an experimental investigation, we explored whether treating temperature data as “language” could provide predictive power. This section describes an approach inspired by Large Language Models (LLMs), where we represent temperatures as ASCII text strings and process them using sequence models with learned embeddings.

5.1 Motivation

Modern language models like GPT achieve remarkable performance by:

1. Tokenizing text into discrete vocabulary elements
2. Learning dense vector embeddings for each token
3. Processing sequences with attention-based architectures
4. Capturing long-range dependencies in sequential data

We asked: *Can we apply similar techniques to temperature time series?* Instead of predicting “the next word,” we use the sequence representation to predict a continuous temperature value.

5.2 Step 1: Tokenization—Temperatures as ASCII Words

The first step converts continuous temperature values into discrete “words” using ASCII string representation.

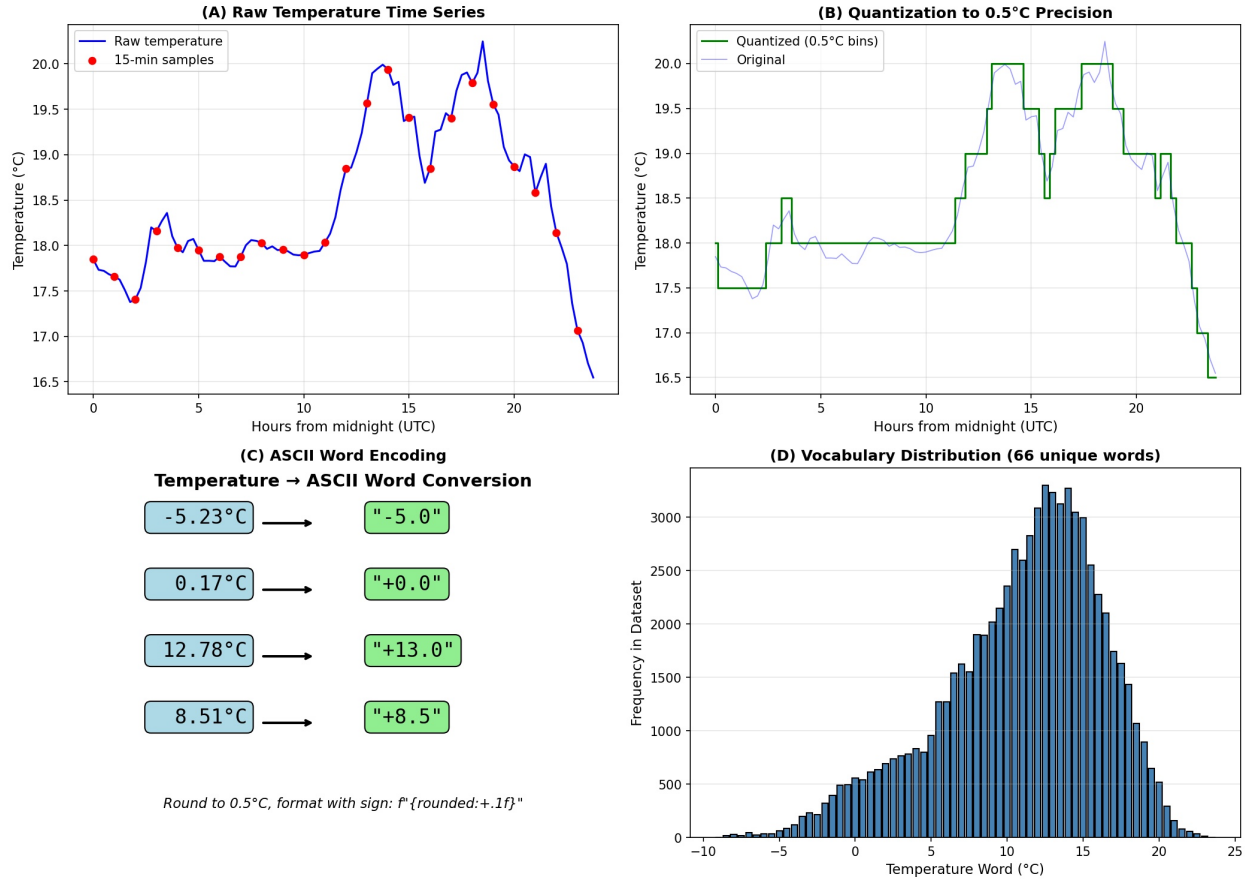


Figure 10: The tokenization process. **(A)** Raw temperature time series at 15-minute intervals. **(B)** Quantization to 0.5°C precision reduces vocabulary size while preserving information. **(C)** Conversion to ASCII strings with explicit sign (e.g., $-5.23^{\circ}\text{C} \rightarrow "-5.0"$, $+12.78^{\circ}\text{C} \rightarrow "+13.0"$). **(D)** Vocabulary distribution showing 68 unique temperature “words” in our dataset.

The tokenization procedure:

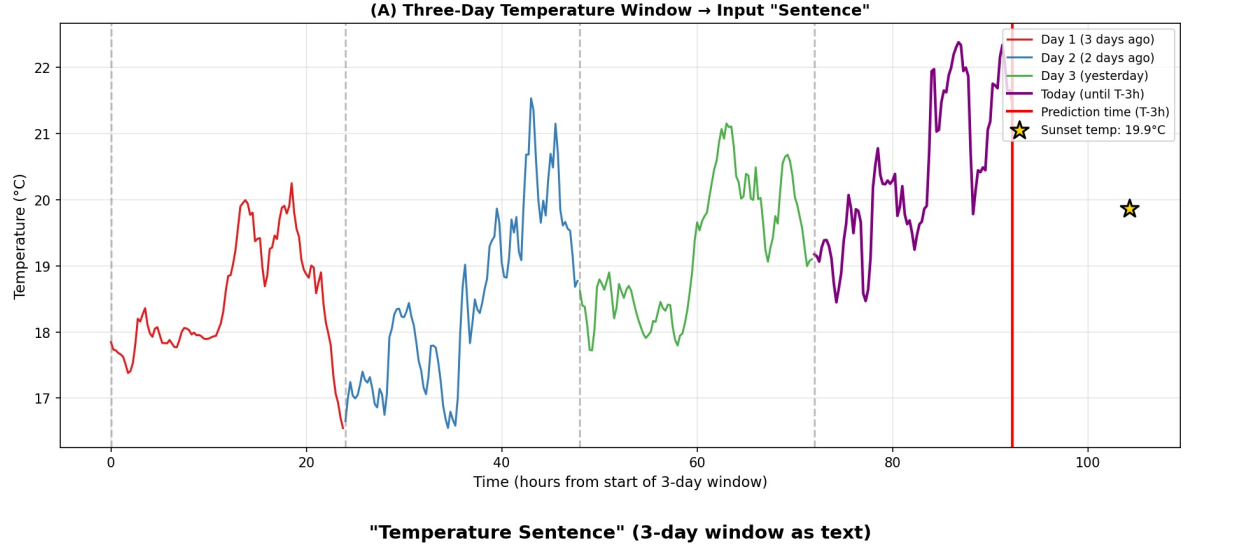
1. **Quantize:** Round temperature to nearest 0.5°C
2. **Format:** Convert to signed ASCII string: `f'{{temp:.1f}}'`

3. **Result:** Vocabulary of 68 unique “words” from “-9.0” to “+23.5”

This is analogous to how text tokenizers convert words to discrete token IDs, except our “words” are temperature strings like “+12.5” or “-3.0”.

5.3 Step 2: Sentence Construction from 3-Day Windows

Each prediction instance becomes a “sentence” composed of all temperature readings from the past 3 days plus the current day up to the prediction time (3 hours before sunset).



+18.0 +17.5 +17.5 +17.5 +17.5 +17.5 +17.5 +17.5 +17.5 +17.5 +18.0 +18.0 +18.0 +18.5 +18.5 ... +21.0 +21.5 +21.5 +21.5 +22.0 +22.5 +22.0 +21.5 +21.5 +21.5

Total sentence length: 370 words

PREDICTION TASK:

Given this "sentence" of 370 temperature words,

predict the sunset temperature: 19.9°C

(Current temp at prediction time: 21.5°C → Predict Δ = -1.6°C)

Figure 11: Construction of temperature “sentences.” **Top:** Three days of historical temperature data plus today’s data up to $T - 3h$ forms the input window. The gold star marks the target sunset temperature we want to predict. **Bottom:** The resulting “sentence” of approximately 371 temperature words that serves as input to the model.

A typical sentence contains ~ 371 words (approximately 93 readings per day $\times$ 4 days). For example:

[‘+8.0’, ‘+8.5’, ‘+9.0’, ‘+9.5’, ..., ‘+11.5’, ‘+12.0’]

This is directly analogous to feeding a sentence of 371 words to a language model.

5.4 Step 3: Model Architecture

Key Insight: Unlike GPT which predicts the probability distribution over the next word, we use the sequence encoding to predict a *continuous* temperature value. This is a regression task, not a classification task.

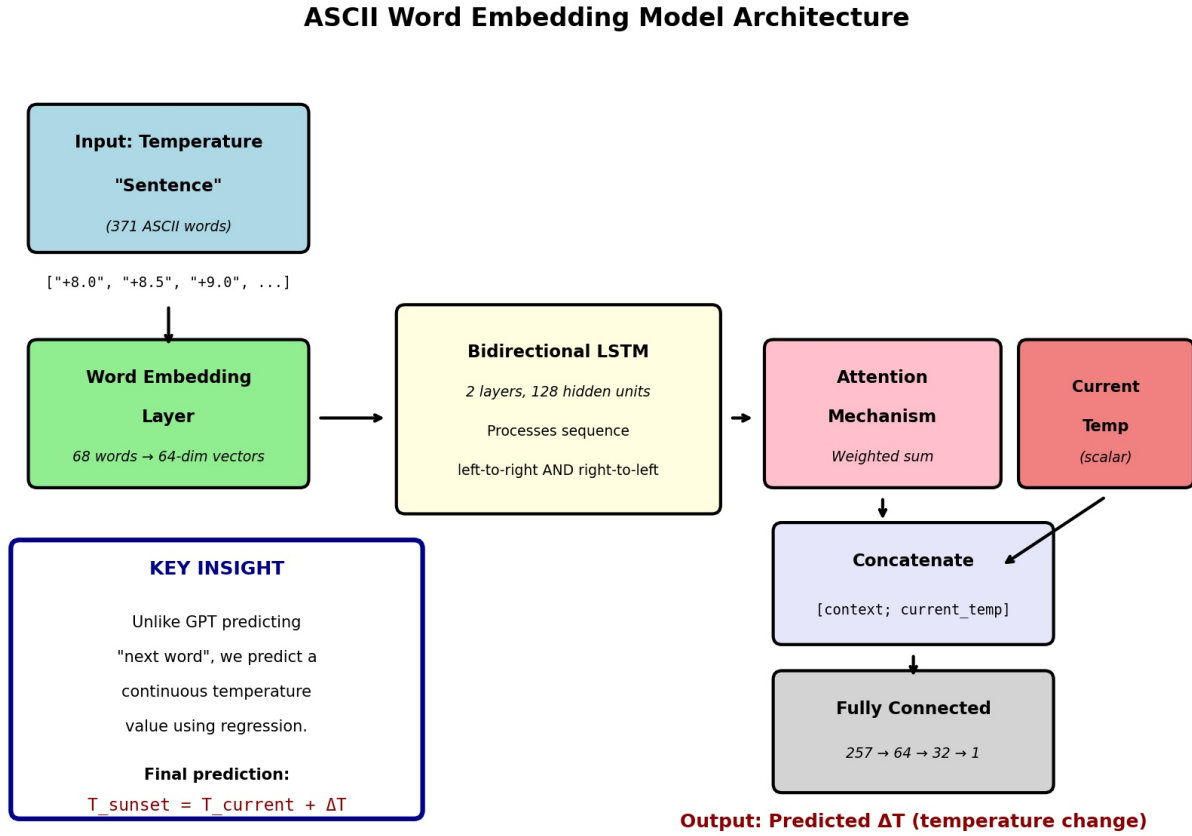


Figure 12: Model architecture for the ASCII word embedding approach. Temperature “words” are embedded into 64-dimensional vectors, processed by a bidirectional LSTM with attention, concatenated with the current temperature, and passed through fully-connected layers to predict the temperature change ΔT . The final prediction is $T_{\text{sunset}} = T_{\text{current}} + \Delta T$.

We implemented two architectures:

1. Bidirectional LSTM with Attention:

- Word embedding layer: 68 words $\rightarrow$ 64-dimensional vectors
- 2-layer bidirectional LSTM (128 hidden units)
- Attention mechanism to weight important time steps
- Concatenate context vector with current temperature
- Fully-connected layers: $257 \rightarrow 64 \rightarrow 32 \rightarrow 1$

2. Transformer Encoder:

- Word + positional embeddings (64-dimensional)

- 3-layer Transformer encoder (4 attention heads)
- Mean pooling over sequence
- Same regression head as LSTM

5.5 Step 4: The Prediction Mechanism

Figure 13 clarifies how prediction works in our approach versus traditional language models.

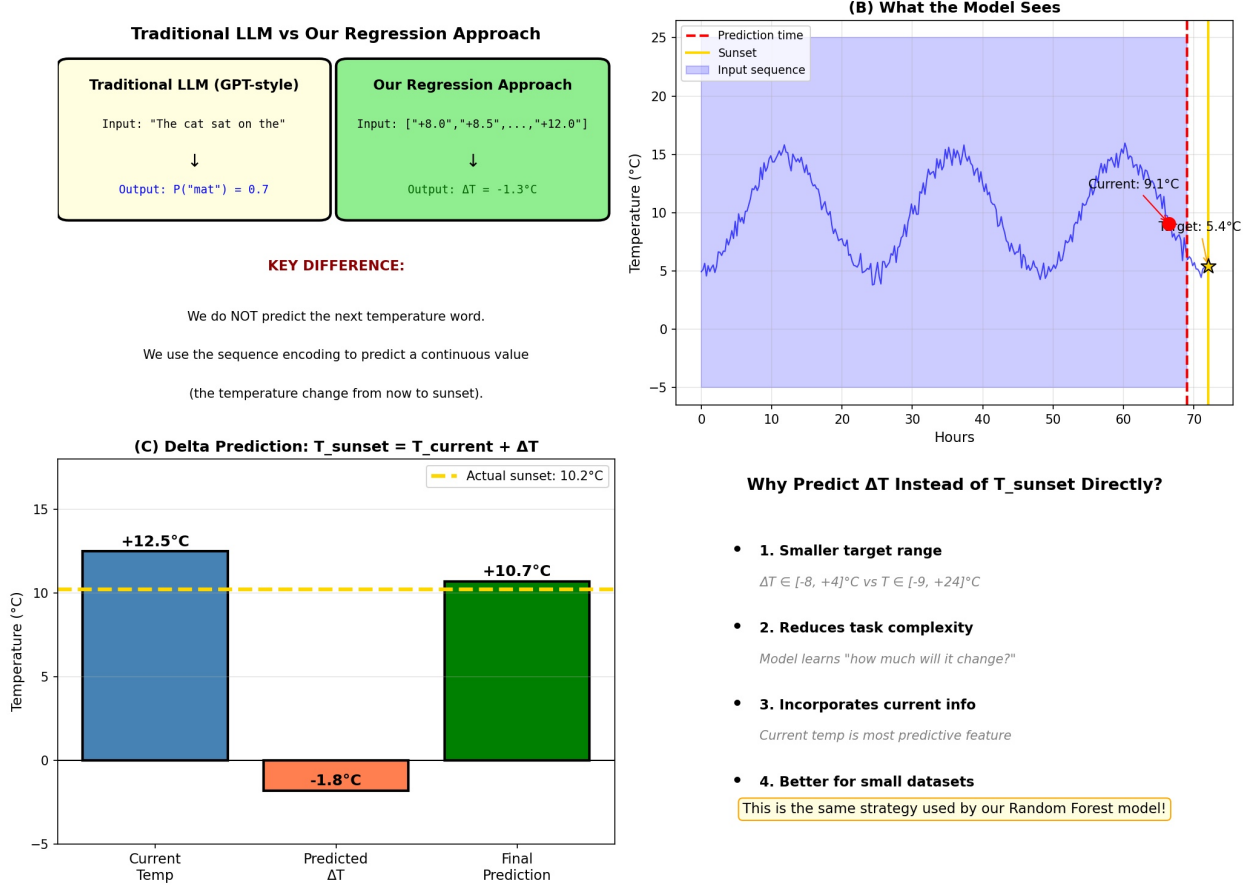


Figure 13: The prediction mechanism explained. **(A)** Comparison with traditional LLMs: we predict a continuous temperature change, not a probability over vocabulary. **(B)** The model sees 3 days of history up to the prediction time (blue shaded region) and must predict the sunset temperature (gold star). **(C)** Delta prediction: the model predicts the temperature *change* from current to sunset, then adds it to the current temperature. **(D)** Rationale for delta prediction.

The model predicts $\Delta T = T_{\text{sunset}} - T_{\text{current}}$, then computes:

$$\hat{T}_{\text{sunset}} = T_{\text{current}} + \Delta T_{\text{predicted}} \quad (1)$$

This delta prediction strategy:

- Reduces target range: $\Delta T \in [-8, +4]^\circ\text{C}$ vs $T \in [-9, +24]^\circ\text{C}$
- Simplifies the learning task: "how much will temperature change?"

- Incorporates the current temperature, our most predictive feature
- Matches the approach used by our successful Random Forest model

5.6 Step 5: Learned Embeddings

The model learns 64-dimensional embeddings for each temperature “word.” Figure 14 visualizes these learned representations.

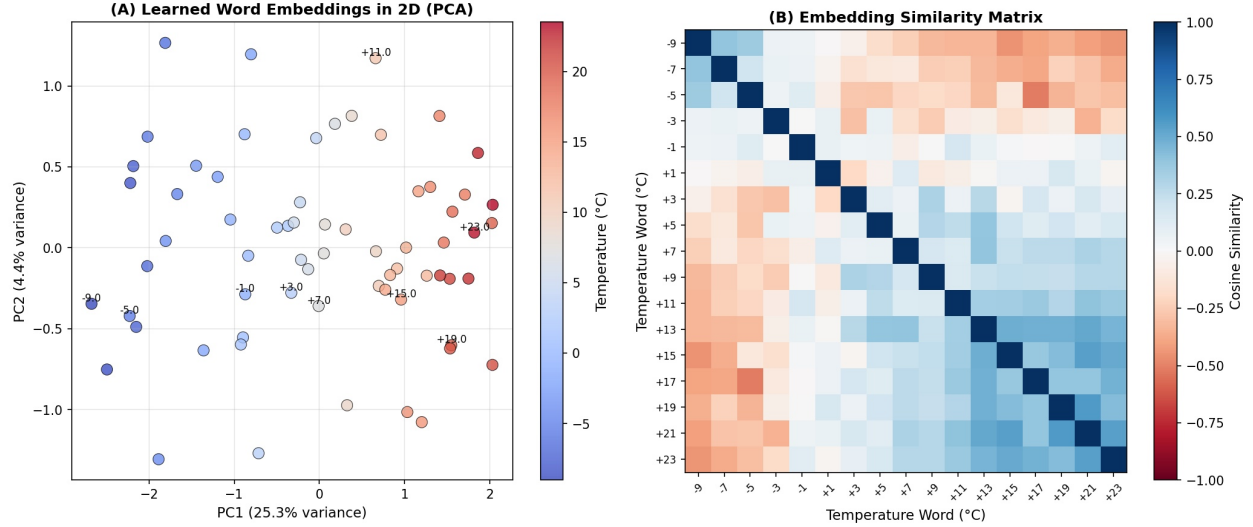


Figure 14: Analysis of learned temperature word embeddings. **(A)** PCA projection of the 64-dimensional embeddings shows that similar temperatures cluster together—the model has learned that “+10.0” and “+10.5” should have similar representations. **(B)** Cosine similarity matrix confirms that nearby temperatures have higher similarity (red diagonal band) while distant temperatures have lower similarity.

The embedding analysis reveals:

- Adjacent temperature words ($\Delta = 0.5^\circ\text{C}$) have positive similarity
- Distant temperature words ($\Delta = 5^\circ\text{C}$) have near-zero or negative similarity
- The model correctly learns the ordinal structure of temperatures

This is analogous to how word embeddings in NLP learn that “king” and “queen” are more similar than “king” and “banana.”

5.7 Results: LLM Approach Performance

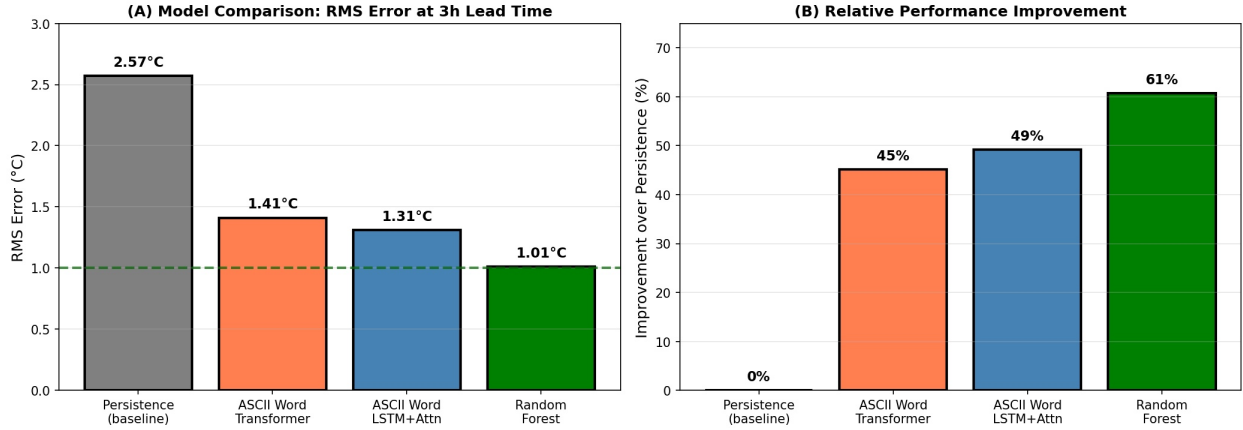


Figure 15: Performance comparison of the LLM-style embedding approach. **(A)** RMS error comparison shows the ASCII word models achieve $\sim 1.3\text{--}1.4^\circ\text{C}$ RMS, outperforming persistence but underperforming Random Forest. **(B)** Improvement over persistence baseline: LSTM achieves 49%, Transformer achieves 45%, while Random Forest achieves 61%.

Table 3: LLM-Style Embedding Approach Results (3-Hour Lead Time)

Model	RMS ($^\circ\text{C}$)	MAE ($^\circ\text{C}$)	Bias ($^\circ\text{C}$)	vs Persistence
Persistence (baseline)	2.57	2.31	+2.24	—
ASCII Word Transformer	1.41	1.13	+0.10	+45%
ASCII Word LSTM+Attention	1.31	1.02	+0.35	+49%
Random Forest	1.01	0.79	−0.04	+61%

5.8 Discussion: Why Random Forest Still Wins

The LLM-style approach achieves meaningful improvements over persistence (45–49%), demonstrating that the “temperature as language” representation can capture predictive patterns. However, Random Forest maintains a significant advantage (61% improvement).

Reasons for Random Forest’s superiority:

1. **Tabular data structure:** Tree-based methods excel at tabular regression with numerical features. The temperature prediction task is fundamentally a structured regression problem, not a sequence-to-sequence task.
2. **Feature engineering:** Random Forest benefits from hand-crafted features (temperature trends, daily statistics, lookback temperatures) that directly encode domain knowledge.
3. **Limited training data:** With only 335 training sentences, the neural models cannot fully leverage their capacity. Language models typically require millions of examples.
4. **Information loss in tokenization:** Quantizing to 0.5°C bins discards fine-grained temperature information that tree models can utilize.

When might LLM approaches excel?

- With significantly more training data (years to decades)
- When incorporating heterogeneous data (text weather reports, satellite imagery)
- For multi-site prediction where transfer learning could help
- With pre-trained foundation models for time series

5.9 Conclusion: LLM Experiment

Treating temperatures as ASCII “words” and using language model architectures is a creative approach that *does work*—achieving 49% improvement over persistence. The models correctly learn that similar temperatures should have similar embeddings. However, for this specific task with limited data and clear numerical structure, traditional tree-based methods remain superior. This experiment demonstrates both the flexibility of modern deep learning and the continued importance of matching model architecture to problem structure.

6 Key Findings

1. **61% improvement over baseline:** Best ML model achieves 1.01°C RMS vs 2.57°C for persistence.
2. **Near-zero bias:** Calibration-based correction reduces systematic bias to <0.05°C.
3. **70% of predictions within 1°C:** At 3 hours, most predictions are highly accurate.
4. **Median error of 0.64°C:** Half of all predictions are within 0.64°C of actual.
5. **72-hour lookback optimal:** Three days of historical data captures diurnal patterns without adding noise.
6. **Tree-based models dominate:** Random Forest and gradient boosting variants outperform neural networks and linear models.

7 Recommendations

7.1 For Operational Deployment

1. Use **Random Forest** with 3-day lookback features
2. Implement **rolling bias correction** using recent prediction errors
3. Retrain monthly to incorporate new data
4. Monitor for anomalous days (frontal passages, unusual weather)

7.2 Potential Future Improvements

1. Incorporate additional meteorological data (cloud cover, pressure, humidity)
2. Implement uncertainty quantification via quantile regression
3. Develop weather-regime-specific models

8 Conclusion

We developed a sunset temperature prediction system achieving **1.01°C RMS error** at 3 hours before sunset with **bias of only -0.04°C** . Using odd calendar days for training and even days for testing ensures rigorous validation. The system achieves a 61% improvement over the persistence baseline and is ready for operational deployment at the Rubin Observatory.

Report generated January 2026. Code available in project repository.